

Docker Development Setup - Raincoat App

Initial Setup (Do Once Per Machine)

1. Install Docker & Docker Compose

- **Mac:** Install Docker Desktop
- **Windows:** Install Docker Desktop
- **Linux:** Install Docker Engine + Docker Compose

2. Create Rails API Application

```
bash

# Create new Rails API app
rails new raincoat_api --api --database=postgresql --skip-docker

# Navigate to project directory
cd raincoat_api
```

3. Add Docker Files

Create the following files in your Rails root directory:

- `docker-compose.yml` (from artifact above)
- `Dockerfile` (from artifact above)
- `init.sql` (from artifact above)

4. Update Gemfile

Add these gems to your Gemfile:

ruby

Gemfile additions

gem 'devise'

gem 'devise-jwt'

gem 'image_processing', '~> 1.2' *# for Active Storage*

gem 'redis', '~> 4.0' *# for caching*

gem 'httparty' *# for API calls*

gem 'neighbor' *# for pgvector*

gem 'google-cloud-vision' *# for AI image processing*

group :development, :test do

gem 'rspec-rails'

end

5. Configure Database

Update `config/database.yml`:

yml

default: &default

adapter: postgresql

encoding: unicode

host: <%= ENV.fetch('DATABASE_HOST', 'localhost') %>

username: <%= ENV.fetch('DATABASE_USERNAME', 'postgres') %>

password: <%= ENV.fetch('DATABASE_PASSWORD', 'password') %>

pool: <%= ENV.fetch("RAILS_MAX_THREADS") { 5 } %>

development:

<<: *default

database: raincoat_development

test:

<<: *default

database: raincoat_test

Daily Development Workflow

Starting Development

```
bash
```

```
# Start all services
```

```
docker-compose up
```

```
# Or run in background
```

```
docker-compose up -d
```

Common Commands

```
bash
```

```
# Run Rails commands
```

```
docker-compose exec rails bundle exec rails generate model User
```

```
docker-compose exec rails bundle exec rails db:migrate
```

```
docker-compose exec rails bundle exec rails console
```

```
# Run tests
```

```
docker-compose exec rails bundle exec rspec
```

```
# Install gems after Gemfile changes
```

```
docker-compose exec rails bundle install
```

```
docker-compose down && docker-compose up --build
```

```
# Database operations
```

```
docker-compose exec rails bundle exec rails db:create
```

```
docker-compose exec rails bundle exec rails db:migrate
```

```
docker-compose exec rails bundle exec rails db:seed
```

Stopping Development

```
bash
```

```
# Stop services
```

```
docker-compose down
```

```
# Stop and remove volumes (fresh start)
```

```
docker-compose down -v
```

Machine Sync Workflow

Switching Between Machines

```
bash
```

```
# Pull latest code
```

```
git pull origin main
```

```
# Ensure you have latest images
```

```
docker-compose pull
```

```
# Start development
```

```
docker-compose up
```

Adding New Dependencies

```
bash
```

```
# After adding gems to Gemfile
```

```
docker-compose exec rails bundle install
```

```
# If gems require native extensions, rebuild
```

```
docker-compose down
```

```
docker-compose up --build
```

Troubleshooting

Database Connection Issues

```
bash
```

```
# Check database status
```

```
docker-compose ps
```

```
# View database logs
```

```
docker-compose logs db
```

```
# Reset database
```

```
docker-compose down -v
```

```
docker-compose up
```

Bundle Install Issues

```
bash
```

```
# Clear bundle cache and rebuild
```

```
docker-compose down
```

```
docker volume rm raincoat_api_bundle_cache
```

```
docker-compose up --build
```

Port Conflicts

If ports 3000, 5432, or 6379 are in use:

```
yaml
```

```
# In docker-compose.yml, change port mappings
```

```
ports:
```

```
- "3001:3000" # Rails
```

```
- "5433:5432" # PostgreSQL
```

```
- "6380:6379" # Redis
```

Next.js Frontend Setup (Optional)

```
bash
```

```
# In separate terminal/directory
```

```
npx create-next-app@latest raincoat-frontend --typescript --tailwind --eslint
```

```
cd raincoat-frontend
```

```
# Configure API URL to point to Rails
```

```
# NEXT_PUBLIC_API_URL=http://localhost:3000
```

Benefits You'll Get

✅ **Identical environment** across all 3 machines ✅ **One-command startup** (`docker-compose up`) ✅

No Ruby/PostgreSQL version conflicts ✅ **pgvector extension** automatically available ✅ **Easy collaboration** if adding team members ✅ **Production parity** for deployment